

Process × File Reference

文件描述符不是文件：引用层次怎样决定 fork、dup、close 和 unlink

408 · OS-B03

Mother Interface

Task → FD Table Entry → Open-File Description → Inode/File Object → Reference/Lifetime Change

本 Bridge 拥有引用关系在不同层级的交接，不重新定义路径解析或文件内容读写。

Owners 与边界

OS01 Owns task 与 per-process fd table；OS05 Owns pathname、inode、open-file state 与生命周期。本册 Owns “一个操作改变哪一层引用”；I/O 与 Page Cache 进入 OS-B04/X-I02。

1 三层对象必须分开

fd 是进程表中的整数索引；fd table entry 指向 open-file description，后者保存当前 offset、status flags 等打开状态；多个 description 可指向同一 inode/file object。fork/dup 复制的是指向关系，不等于复制文件内容。

2 操作对照

操作	主要新增/共享关系	常见后果
fork	子表复制，条目指向同一 OFD	父子共享 offset
dup	同一进程新增 fd，指向同一 OFD	close 一个不必关闭 OFD
unlink	移除目录名，保留已有引用	最后引用消失后才可回收
close	移除一个 fd table entry	可能触发 OFD/inode 引用递减

3 最小例子与 Anti-Bridge

父进程打开文件后 fork，父子交替 read；由于共享 OFD，文件 offset 可能共同推进。若重新 open，则得到新的 OFD，offset 不共享。fd ≠ inode；unlink ≠ 立即删除已有打开对象；close ≠ 必然销毁文件内容。

Compression

题目给的是哪个对象？操作复制的是索引、OFD 还是 inode 引用？哪个字段共享？最后一个引用何时消失？